Queen Mary
University of London
Science and Engineering

EBU4202: Digital Circuit Design
Karnaugh Maps

Dr. Md Hasanuzzaman Sagor (Hasan)
Dr. Chao Shu (Chao)
Dr. Farha Lakhani (Farha)
Dr. Fatma Benkhelifa

School of Electronic Engineering and Computer Science,
Queen Mary University of London,
London, United Kingdom.

Relationships: F, F′, Σ, Π

Karnaugh Map for F		Karnaugh Map for F′	
Minimal Sum of Products for F	**Minimal Product of Sums for F**	**Minimal Sum of Products for F′**	**Minimal Product of Sums for F′**
What to do: Loop *1's* and read as *minterms*.	***What to do***: Loop *0's* and read as *maxterms* (or complement SOP for *F*).	***What to do***: Loop *1's*.	***What to do***: Loop *0's* and read as *maxterms* (or complement SOP for *F*).

Relationships: F, F′, Σ, Π

Find the SoP and PoS for the following K-Map.

AB \ CD	00	01	11	10
00	1	1	1	1
01	1	0	0	1
11	1	0	0	0
10	1	1	0	0

Relationships: F, F′, Σ, Π

Find the SoP and PoS for the following K-Map.

AB \ CD	00	01	11	10
00	1	1	1	1
01	1	0	0	1
11	1	0	0	0
10	1	1	0	0

SoP, F = A'B' + C'D' + B'C' + A'D'

Relationships: F, F′, Σ, Π

Find the SoP and PoS for the following K-Map.

AB \ CD	00	01	11	10
00	1	1	1	1
01	1	0	0	1
11	1	0	0	0
10	1	1	0	0

Relationships: F, F′, Σ, Π

Find the SoP and PoS for the following K-Map.

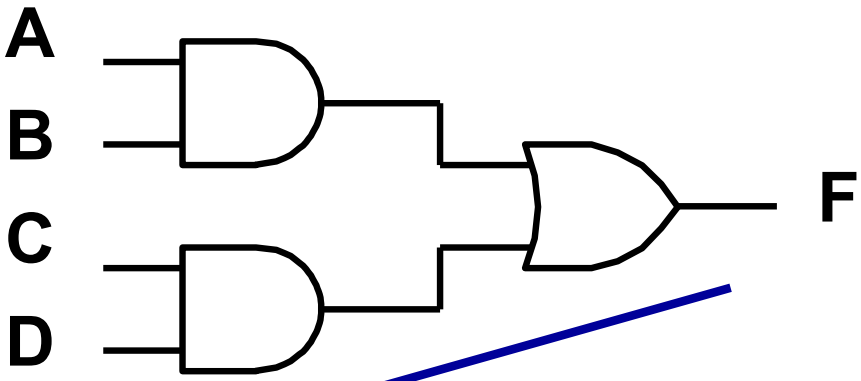
AB \ CD	00	01	11	10
00	1	1	1	1
01	1	0	0	1
11	1	0	0	0
10	1	1	0	0

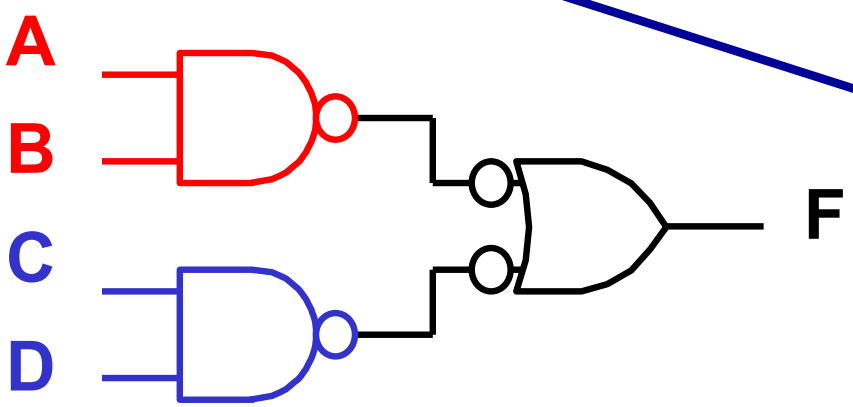
POS F = (B’ + D’)(A’ + C’)

Converting to Negative Logic

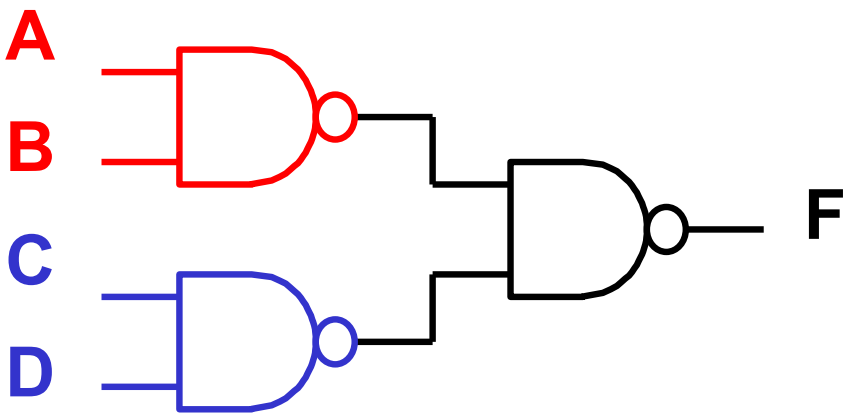
- Three different (but *equivalent*) circuits that implement the equation **F = AB + CD**:

What are the equations describing the two bottom diagrams?





F = ((AB)')' + ((CD)')'



F = ((AB)'.(CD)')'

Example: SOP of F (1/2)

- Find the *minimal sum of products* and draw the *NAND gate* implementation for:

$$F = X'Y'Z + X'YZ' + X'YZ + XY'Z' + XY'Z$$

- ***What to do***:
  - ✓ First look for the largest possible group.
  - ✓ If there are still *1's* to cover, keep circling until all are covered.
  - ✓ Read off essential prime implicants necessary for minimal cover.

X \ YZ	00	01	11	10
0	0	1	1	1
1	1	1	0	0

$$F = XY' + X'Y + Y'Z$$

$$F = F'' = (XY' + X'Y + Y'Z)''$$

$$F = ((XY')'(X'Y)'(Y'Z)')'$$

Example: SOP of F (2/2)

To be completed in class …

- Draw the *NAND gate* implementation of: $F = XY' + X'Y + Y'Z$

✓ 1. Draw the normal circuit:

✓ 2. Apply De Morgan's theorems graphically:

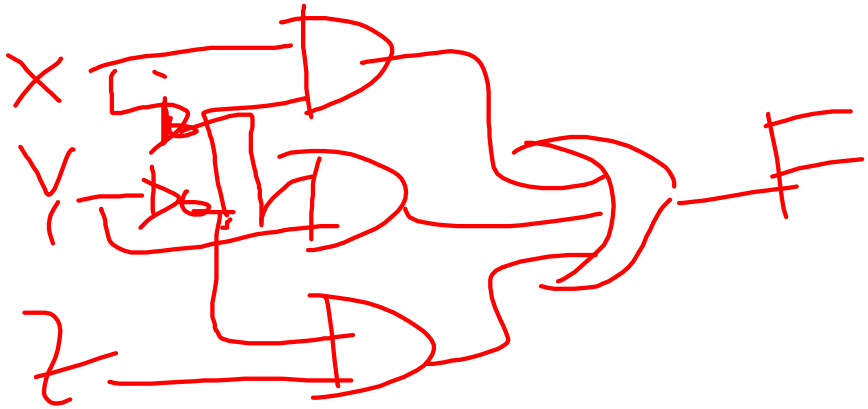
✓ 3. Convert the 3-inverted input gate to *NAND*:

Example: SOP of F (2/2)

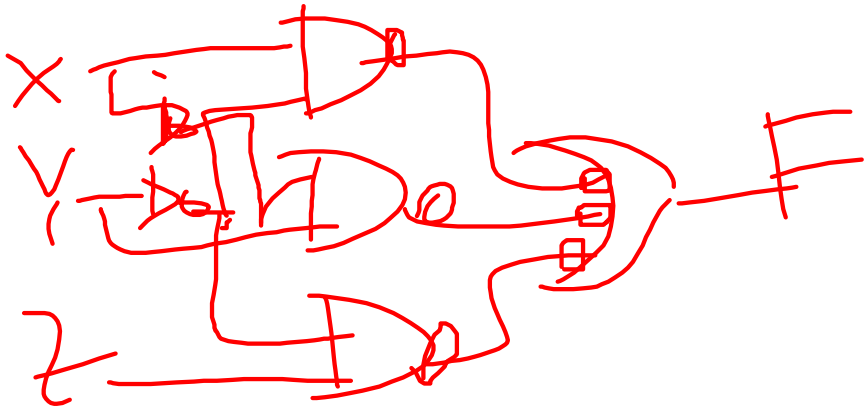
To be completed in class …

- Draw the *NAND gate* implementation of: F = XY′ + X′Y + Y′Z

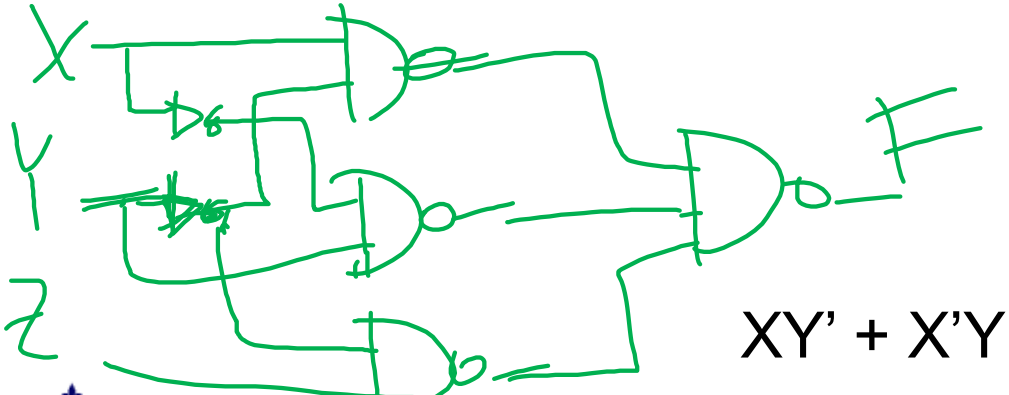
✓ 1. Draw the normal circuit:



✓ 2. Apply De Morgan's theorems graphically:



✓ 3. Convert the 3-inverted input gate to *NAND*:



XY' + X'Y + Y'Z = ((XY')')' + ((X'Y)')' + ((Y'Z)')'

= [(XY')'.(X'Y)'.(Y'Z)']'

Example: SOP of F′

To be completed in class …

- Find the *minimal sum of products* for F′ where,

  $F = X'Y'Z + X'YZ' + X'YZ + XY'Z' + XY'Z$

  $\therefore\ F' = (X+Y+Z')(X+Y'+Z)(X+Y'+Z')(X'+Y+Z)(X'+Y+Z')$

- ***What to do***:
  - ✓ Map F′.
  - ✓ Circle the *1's*.
  - ✓ Read off essential prime implicants as *minterms*.

F is as in previous example.

X \ YZ	00	01	11	10
0				
1				

Example: SOP of F′

To be completed in class …

- Find the *minimal sum of products* for F′ where,

  F = X′Y′Z + X′YZ′ + X′YZ + XY′Z′ + XY′Z

  ∴ F′ = (X+Y+Z′)(X+Y′+Z)(X+Y′+Z′)(X′+Y+Z)(X′+Y+Z′)

- ***What to do***:
  - ✓ Map F′.
  - ✓ Circle the *1's*.
  - ✓ Read off essential prime implicants as *minterms*.

F is as in previous example.

X \ YZ	00	01	11	10
0	1	0	0	0
1	0	0	1	1

F′ = XY + X'Y'Z'

Summary: Karnaugh Maps (1/2)

- Karnaugh maps are an *alternative* to Switching Algebra for *simplifying expressions*:
  - The result is an MSP (or MPS) that leads to a minimal 2-level circuit.
  - It is easy to handle ***don't-care conditions***.
  - Karnaugh maps are really only good for manual simplification of fairly small expressions.

We'll look at these **in a few slides**.

Summary: Karnaugh Maps (2/2)

- Things to keep in mind:
  - Remember the correct order of *minterms* on the K-map.
  - When grouping, it is possible to wrap around all sides of the K- map, and the groups can overlap.
  - Make as few groups (of adjacent *1s* or *0s*) as possible, but <u>make each of them as large as possible</u> (*only in powers of 2* i.e., *1*, *2*, *4*, *8*, …).
  - There may be more than one valid solution!!

Map Manipulation

- Often a truth table will be generated that has several combinations that the designer doesn't care about.
- **Expressions can be simplified by using DON'T CARE cases**:
  - Sometimes it is possible to eliminate essential prime implicants.
- **Example**:
  - Consider the following *incompletely specified function*:

$$\begin{cases} F(W,X,Y,Z) = \Sigma m(1, 3, 7, 11, 15) \\ d(W,X,Y,Z) = \Sigma m(0, 2, 5) \rightarrow \text{These are DON'T CARE cases.} \end{cases}$$

4-Variable Karnaugh Map

W	X	Y	Z	F
0	0	0	0	x
0	0	0	1	1
0	0	1	0	x
0	0	1	1	1
0	1	0	0	0
0	1	0	1	x
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

$F(W,X,Y,Z) = \Sigma m(1, 3, 7, 11, 15)$
$d(W,X,Y,Z) = \Sigma m(0, 2, 5)$

WX \ YZ	00	01	11	10
00	x	1	1	x
01	0	x	1	0
11	0	0	1	0
10	0	0	1	0

Answer:
F = YZ + W'X'

4-Variable Karnaugh Map

W	X	Y	Z	F
0	0	0	0	x
0	0	0	1	1
0	0	1	0	x
0	0	1	1	1
0	1	0	0	0
0	1	0	1	x
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

$F(W,X,Y,Z) = \Sigma m(1, 3, 7, 11, 15)$
$d(W,X,Y,Z) = \Sigma m(0, 2, 5)$

WX \ YZ	00	01	11	10
00	x	1	1	x
01	0	x	1	0
11	0	0	1	0
10	0	0	1	0

Alternative Answer:

F =

4-Variable Karnaugh Map

W	X	Y	Z	F
0	0	0	0	x
0	0	0	1	1
0	0	1	0	x
0	0	1	1	1
0	1	0	0	0
0	1	0	1	x
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

$F(W,X,Y,Z) = \Sigma m(1, 3, 7, 11, 15)$
$d(W,X,Y,Z) = \Sigma m(0, 2, 5)$

WX \ YZ	00	01	11	10
00	x	1	1	x
01	0	x	1	0
11	0	0	1	0
10	0	0	1	0

Alternative Answer:
F = YZ + W'Z

Example 2: Don't-Care Cases - PoS

- Design a circuit that will produce an output 1 if the BCD input combination is an even number. The BCD input combination 0000 is not allowed to occur.

(1) Find the PoS using the Karnaugh map.

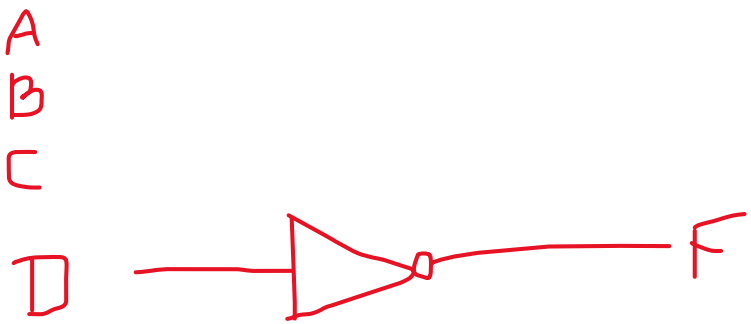
(2) Find the circuit diagram

#	A	B	C	D	F(A,B,C, D)
0	**0**	0	0	0	
1	**0**	0	0	1	
2	**0**	0	1	0	
3	**0**	0	1	1	
4	**0**	1	0	0	
5	**0**	1	0	1	
6	**0**	1	1	0	
7	**0**	1	1	1	
8	**1**	0	0	0	
9	**1**	0	0	1	
10	**1**	0	1	0	
11	**1**	0	1	1	
12	**1**	1	0	0	
13	**1**	1	0	1	
14	**1**	1	1	0	
15	**1**	1	1	1	

Example 2: Don't-Care Cases - PoS

AB \ CD	00	01	11	10
00	x	0	0	1
01	1	0	0	1
11	x	x	x	x
10	1	0	x	x

$$F = D'$$



#	A	B	C	D	F(A,B,C,D)
0	0	0	0	0	X
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	0
4	0	1	0	0	1
5	0	1	0	1	0
6	0	1	1	0	1
7	0	1	1	1	0
8	1	0	0	0	1
9	1	0	0	1	0
10	1	0	1	0	X
11	1	0	1	1	X
12	1	1	0	0	X
13	1	1	0	1	X
14	1	1	1	0	X
15	1	1	1	1	X

Example 3: Don't-Care Cases - SoP

Simplify:

$F(A, B, C, D) = \Sigma m(1, 3, 5, 7, 9) + d(6, 12, 13)$

AB \ CD	00	01	11	10
00	0	1	1	0
01	0	1	1	x
11	x	x	0	0
10	0	1	0	0

Example 3: Don't-Care Cases - SoP

Simplify:

$F(A, B, C, D) = \Sigma m(1, 3, 5, 7, 9) + d(6, 12, 13)$

AB \ CD	00	01	11	10
00	0	1	1	0
01	0	1	1	x
11	x	x	0	0
10	0	1	0	0

AB \ CD	00	01	11	10
00	0	1	1	0
01	0	1	1	x
11	x	x	0	0
10	0	1	0	0

Without using don't cares:

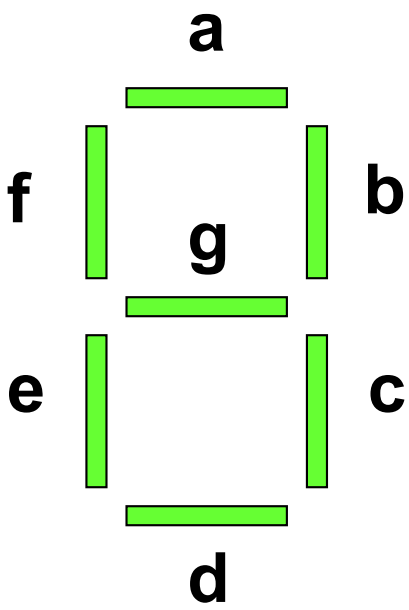
F= A'D + B'C'D

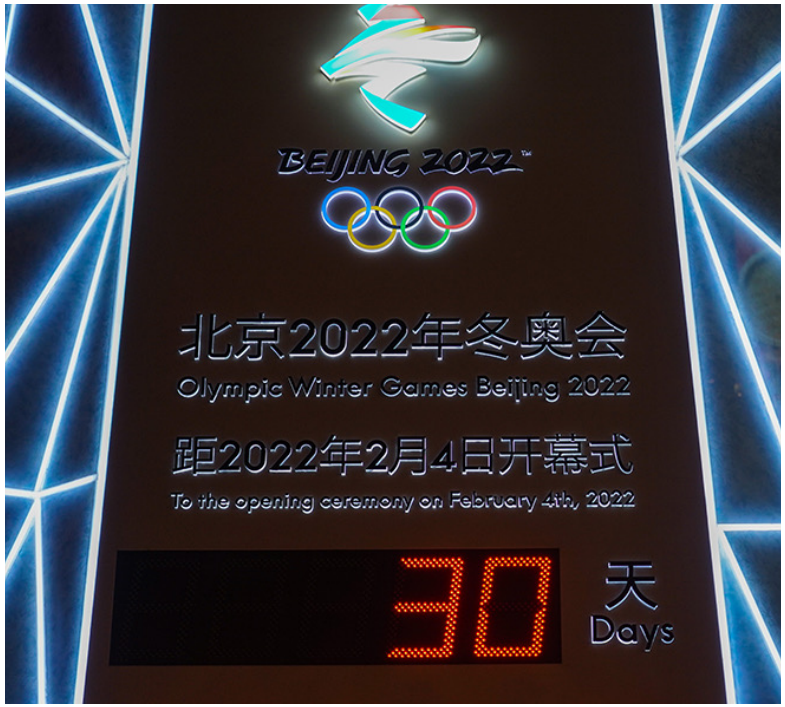
With using don't cares:

F= A'D + C'D

Example (1/9): Design Procedure

- **Problem Specification**:
  - *Design a code converter* to decode from BCD to a 7-segment display.
  - The combinational circuit must accept a BCD digit *and* generate the appropriate outputs to display the digit in a 7-segment format as shown here.



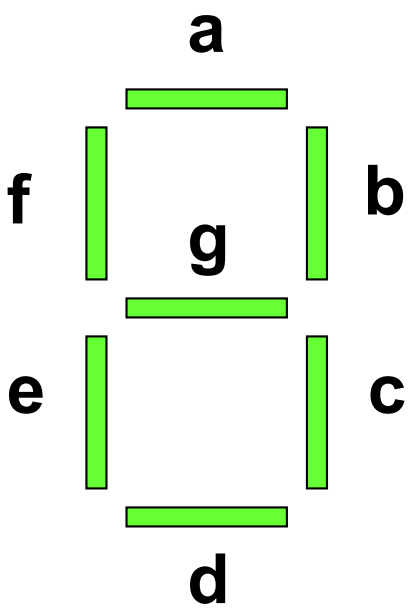


Example (2/9): BCD and I/O

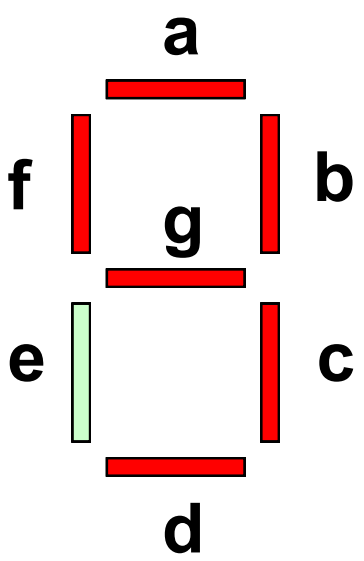
- *BCD numbers are 4 digits long*, so
  - e.g., $1_{10} = 0001_2$ and $10_{10} = 00010000_2$.

- **Inputs**:
  - We only have to output decimal digits *0-9* on the 7-segment display, so only 4 inputs are needed!

- **Outputs**:
  - Need one for each segment *a-g*.
  - Output for each function *a-g* should be:
    - *1* if that segment should be *on*.
    - *0* if it should be *off*.



Example (3/9): BCD 7-Segment Truth Table

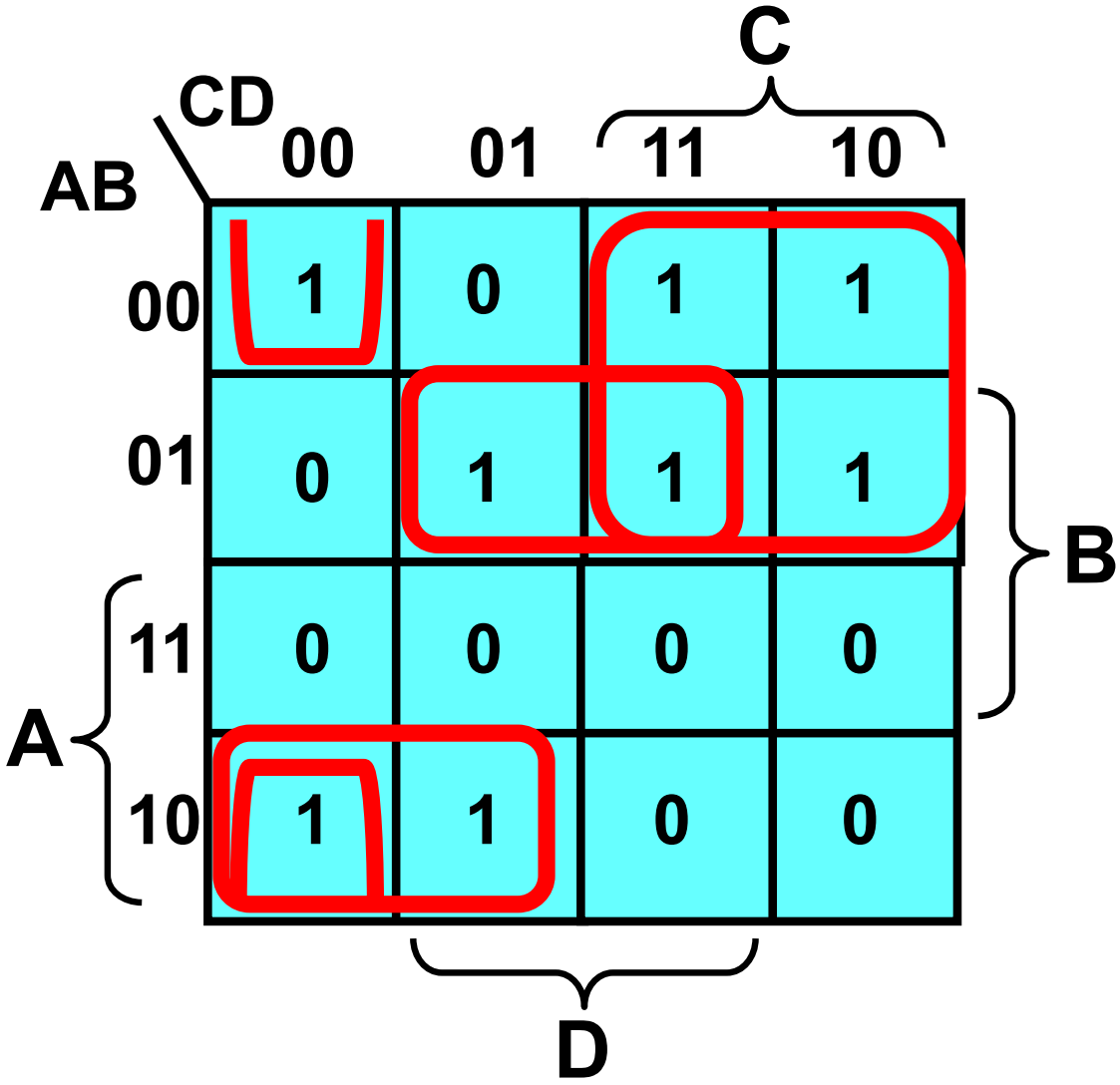


	A	B	C	D	a	b	c	d	e	f	g
0 ⇨	0	0	0	0	1	1	1	1	1	1	0
1 ⇨	0	0	0	1	0	1	1	0	0	0	0
2 ⇨	0	0	1	0	1	1	0	1	1	0	1
3 ⇨	0	0	1	1	1	1	1	1	0	0	1
4 ⇨	0	1	0	0	0	1	1	0	0	1	1
5 ⇨	0	1	0	1	1	0	1	1	0	1	1
6 ⇨	0	1	1	0	1	0	1	1	1	1	1
7 ⇨	0	1	1	1	1	1	1	0	0	0	0
8 ⇨	1	0	0	0	1	1	1	1	1	1	1
9 ⇨	1	0	0	1	1	1	1	1	0	1	1
	1	0	1	0	0	0	0	0	0	0	0
	1	0	1	1	0	0	0	0	0	0	0
	1	1	0	0	0	0	0	0	0	0	0
	1	1	0	1	0	0	0	0	0	0	0
	1	1	1	0	0	0	0	0	0	0	0
	1	1	1	1	0	0	0	0	0	0	0

Example (4/9): Karnaugh Map for ‘a’

$a = B'C'D' + AB'C' + A'BD + A'C$

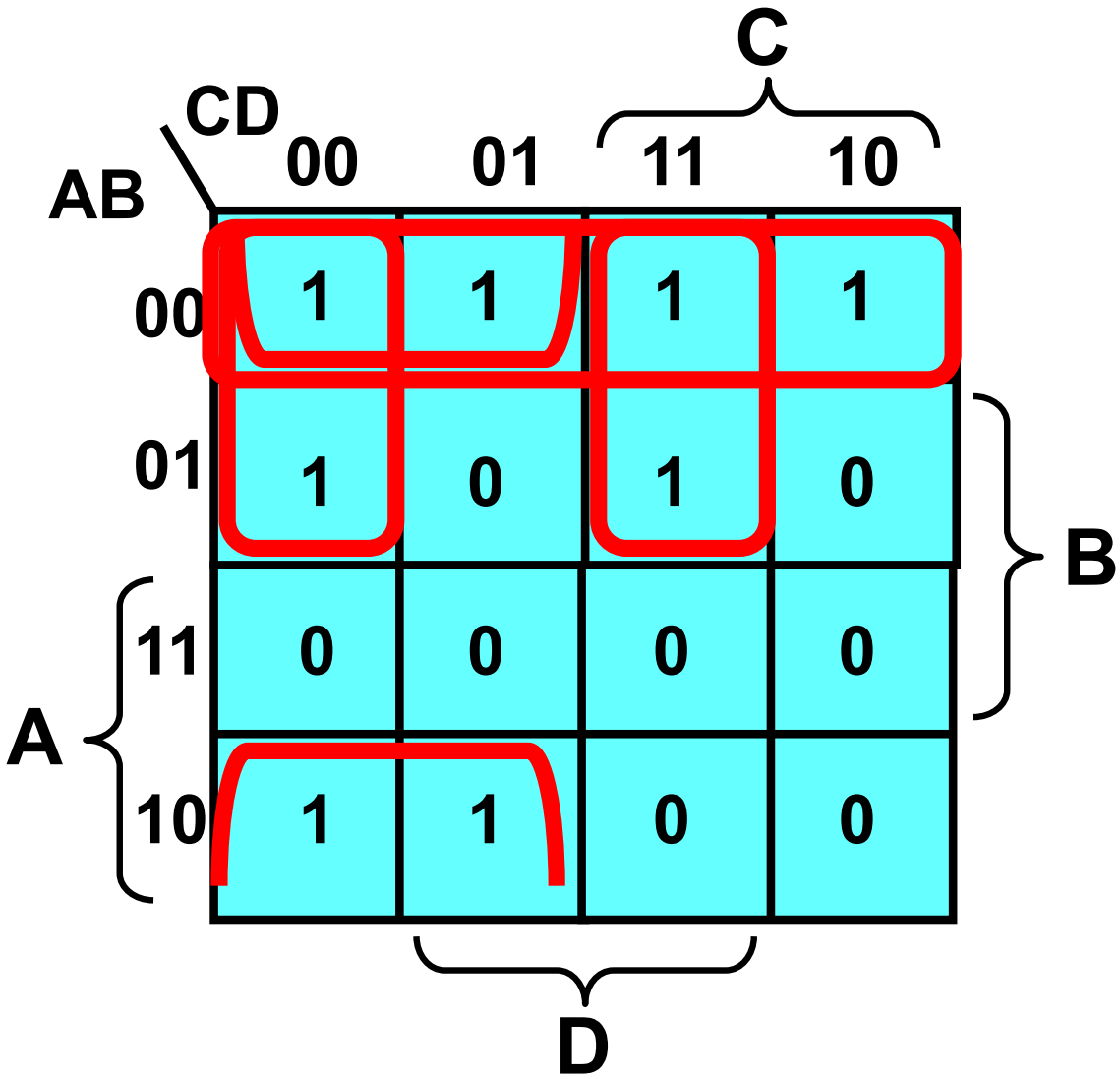
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0



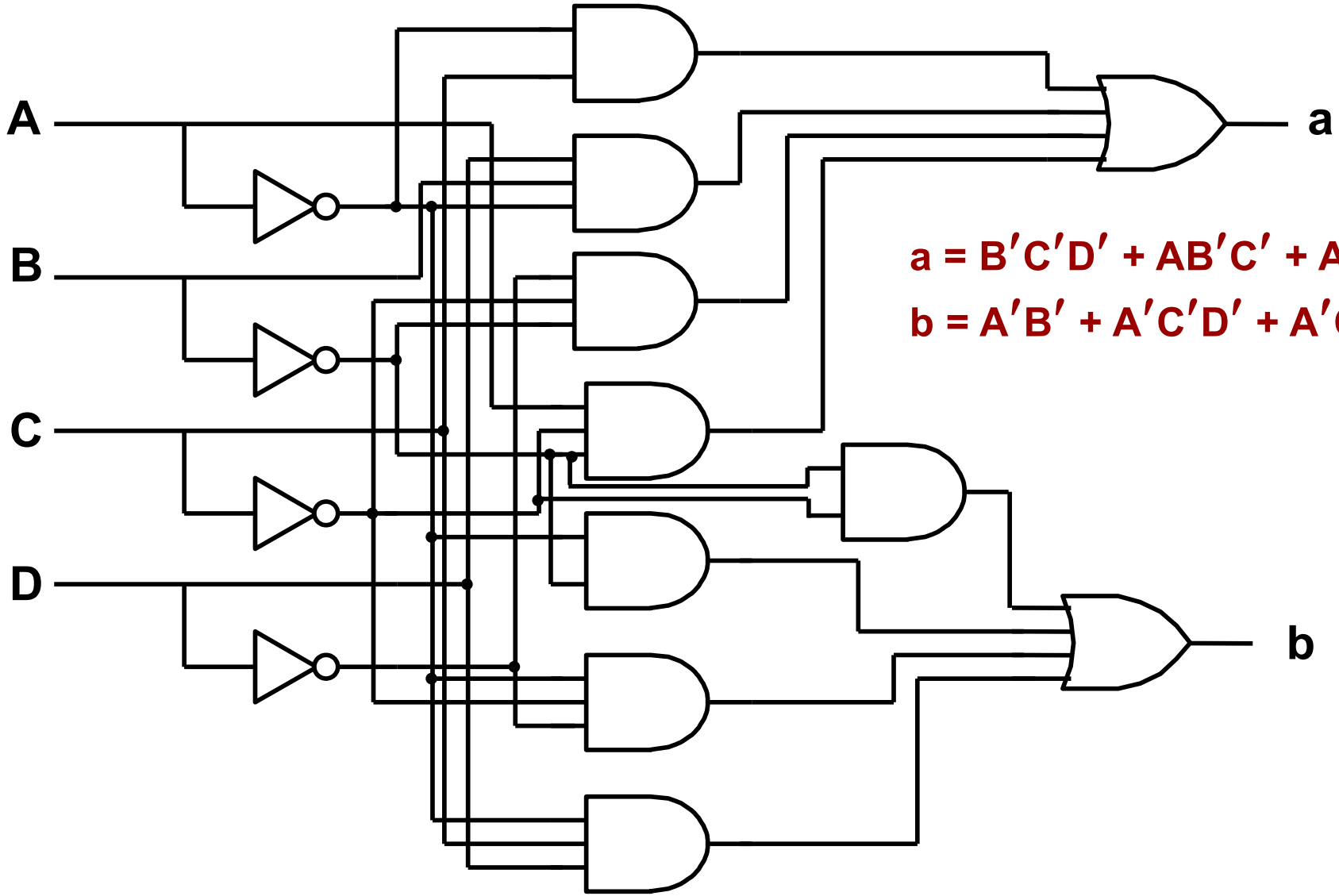
Example (5/9): Karnaugh Map for ‘b’

$b = A'B' + A'C'D' + A'CD + B'C'$

A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0



Example (6/9): BCD 7-Segment Decoder (for *a* & *b*)



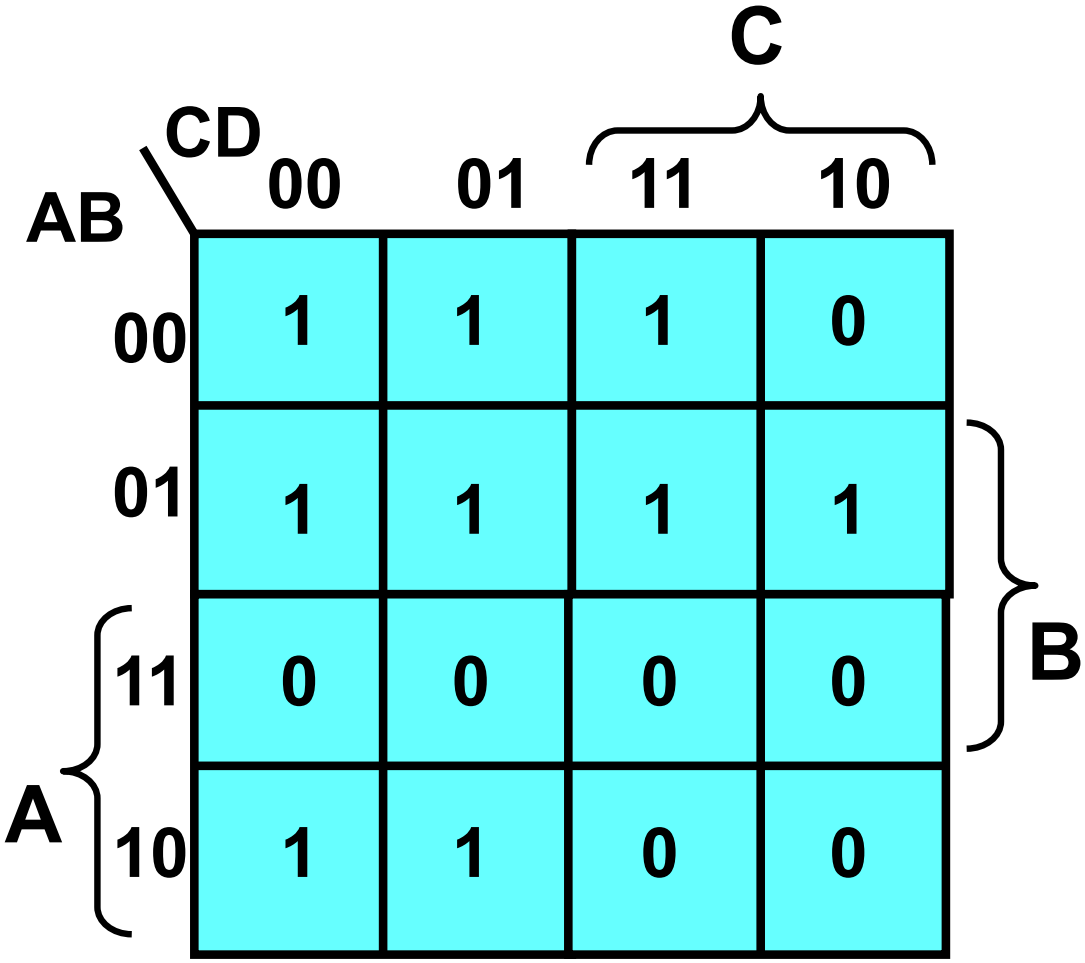
$a = B'C'D' + AB'C' + A'BD + A'C$

$b = A'B' + A'C'D' + A'CD + B'C'$

Example (7/9): Karnaugh Map for ‘c’

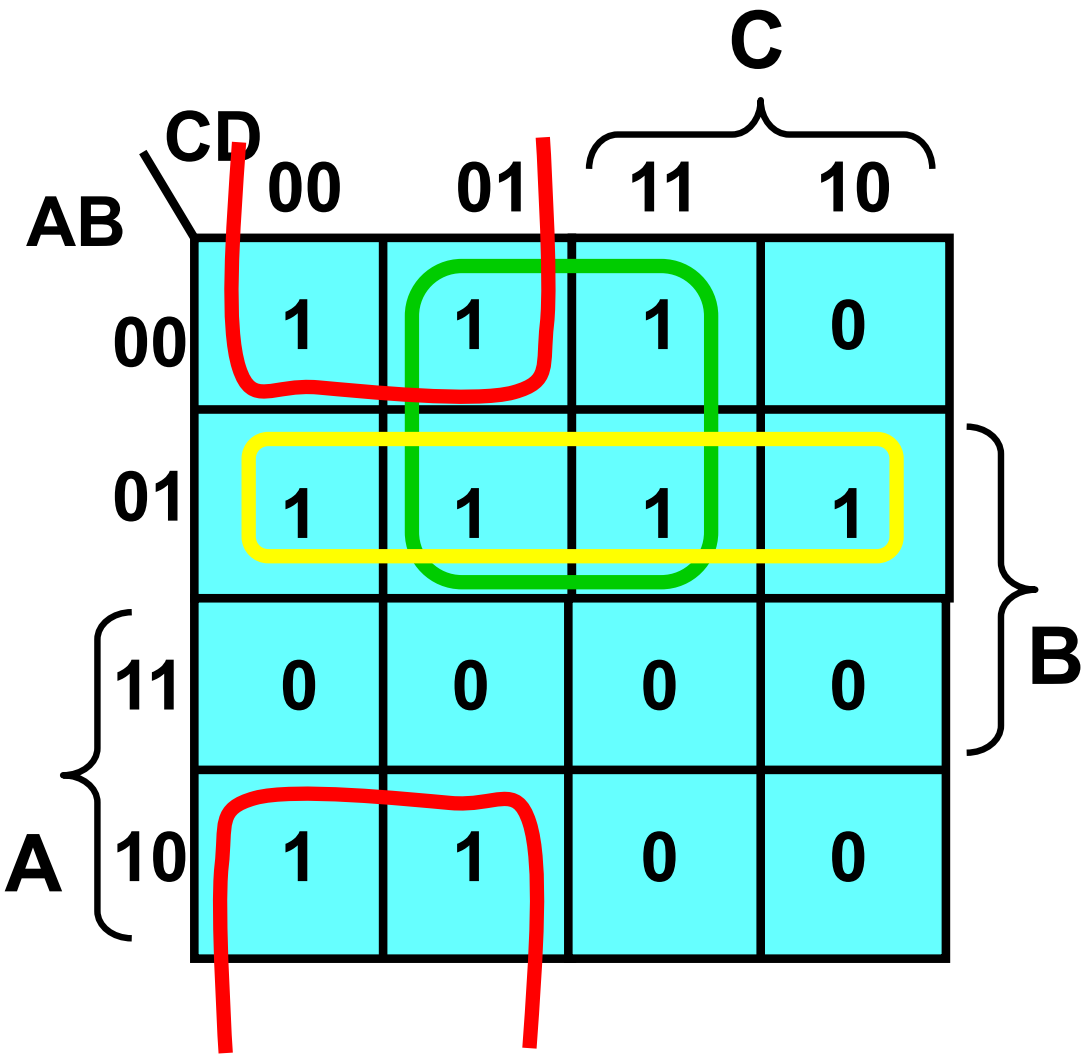
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

c =



Example (7/9): Karnaugh Map for ‘c’

A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0



c = B’C’ + A’D + A’B

EPI

Example (8/9): Karnaugh Map for ‘d’

d =

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	0	0	0	0
10	1	1	0	0

C

B

A

D

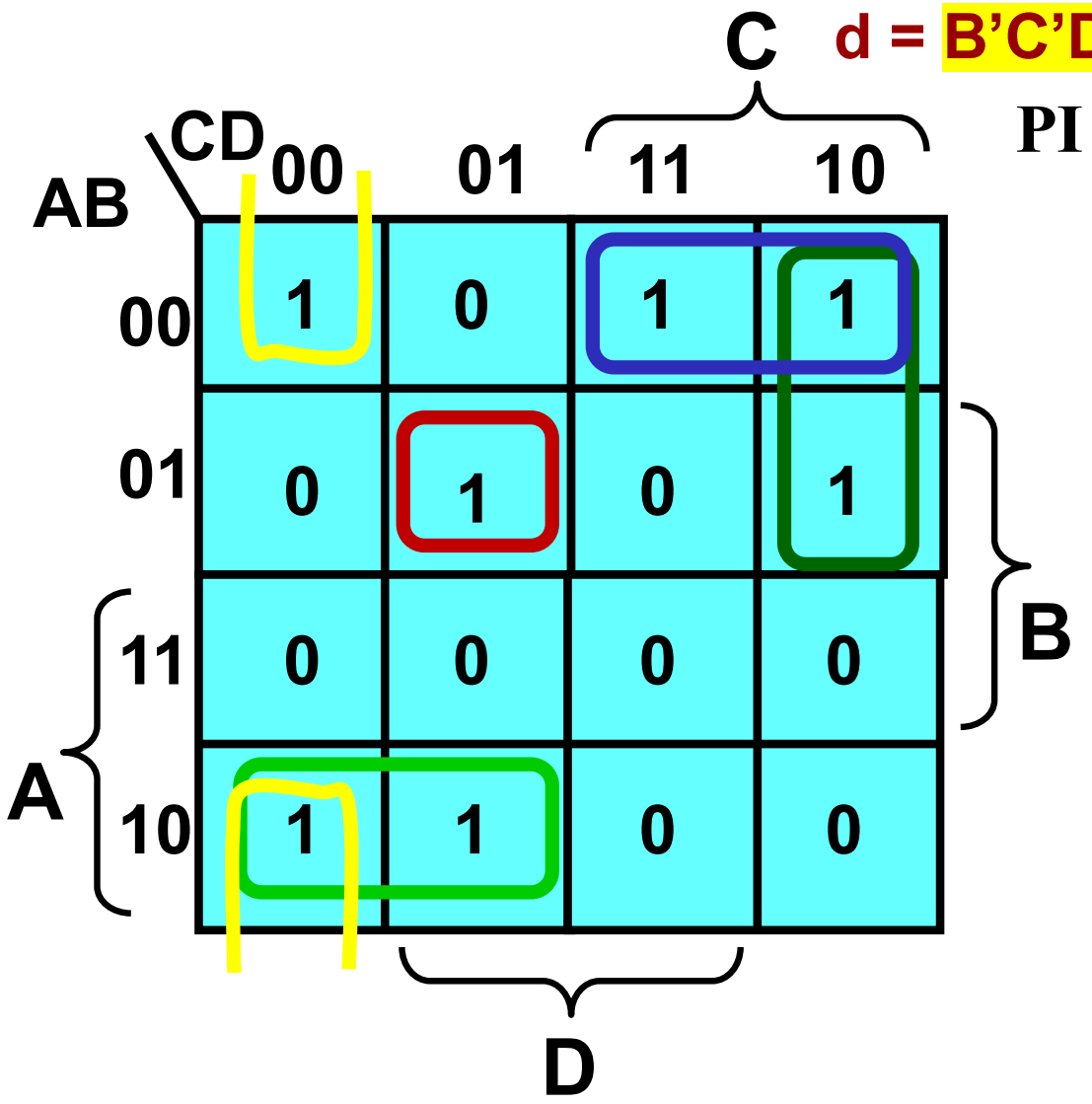
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

Example (8/9): Karnaugh Map for ‘d’

$d = B'C'D' + AB'C' + A'B'C + A'CD' + A'BC'D$

PI

EPI



A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

Example (1/2): Parity Generators

- **Problem Specification**: **Build a network** that will generate the appropriate even parity bit for the 3-bit input.
  - *Inputs*: X = Bit1; Y = Bit2; Z = Bit3
  - *Outputs*: F = parity bit

Remember: In a 4-bit number, there will be *even parity*, if there is an even number of *1's*.

What is the requirement for having *odd parity*?

Example (2/2): 3-Bit Even Parity Generator

To be completed in class …

Truth Table

X	Y	Z	F
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

X \ YZ	00	01	11	10
0				
1				

Example (2/2): 3-Bit Even Parity Generator

To be completed in class …

Truth Table

X	Y	Z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

X \ YZ	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Example 2:
3-Bit Even Parity Generator

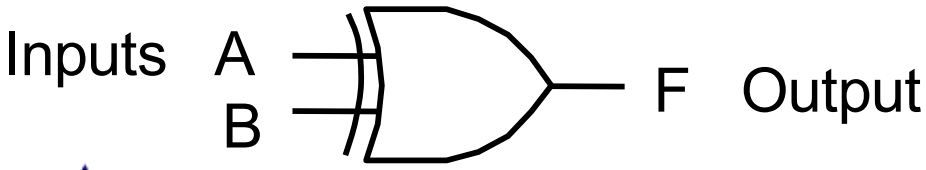
To be completed in class …

Truth Table

X	Y	Z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

X \ YZ	00	01	11	10
0	0	1	0	1
1	1	0	1	0

$$\mathbf{F} = XY'Z' + XYZ + X'Y'Z + X'YZ'$$
$$= X(Y'Z' + YZ) + X'(Y'Z + YZ')$$
$$= X(Y \oplus Z)' + X'(Y \oplus Z)$$
$$= \mathbf{X \oplus Y \oplus Z}$$



A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

$F = (A \oplus B)$ *or*

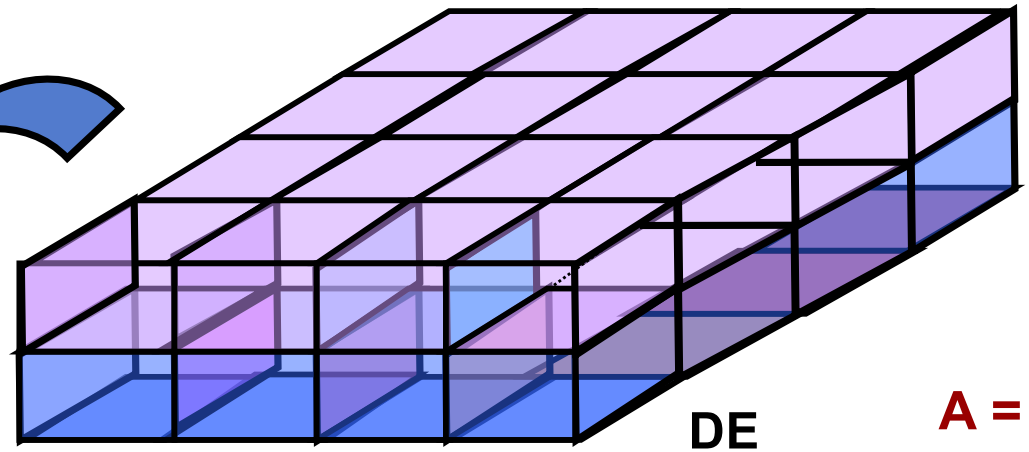
$F = A'B + AB'$ *or*

$F = \bar{A}B + A\bar{B}$

$F' = AB + A'B'$

Queen Mary University of London

5-Variable Karnaugh Map



A = 0

BC \ DE	00	01	11	10
00	m0	m1	m3	m2
01	m4	m5	m7	m6
11	m12	m13	m15	m14
10	m8	m9	m11	m10

Each square in ***A=0*** is also adjacent to squares in ***A=1***.

A = 1

BC \ DE	00	01	11	10
00	m16	m17	m19	m18
01	m20	m21	m23	m22
11	m28	m29	m31	m30
10	m24	m25	m27	m26

Example (1/2): 5-Variable Karnaugh Map

F(A,B,C,D,E) = Σm(0, 10, 11, 14, 15, 16, 20, 24, 26, 27, 28, 30, 31)

= 00000, 01010, 01011, 01110, 01111, 10000, 10100
11000, 11010, 11011, 11100, 11110, 11111

A = 0

BC \ DE	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	0	0	1	1
10	0	0	1	1

A = 1

BC \ DE	00	01	11	10
00	1	0	0	0
01	1	0	0	0
11	1	0	1	1
10	1	0	1	1

Example (2/2): 5-Variable Karnaugh Map

Next step: Grouping ... Start with largest possible – in this map it's *32*, down to *1*. Don't circle smaller groups if they don't include uncovered *1's*!

Cross-map (i.e., A=0 and A=1) so no A; but every other variable is the same, so B'C'D'E'.

Only in map *A=1*; *B* and *C* change, so AD'E'.

A = 0

BC \ DE	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	0	0	1	1
10	0	0	1	1

Remember the cross-map adjacency!

A = 1

BC \ DE	00	01	11	10
00	1	0	0	0
01	1	0	0	0
11	1	0	1	1
10	1	0	1	1

$$F = B'C'D'E' + AD'E' + BD$$

Cross-map (i.e., *A=0* and *A=1*) so no *A*; in both maps *C* and *E* change, so *BD*.

Find the SoP from the 5-variable K-map.

A = 0

BC \ DE	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	1	1	0
10	0	1	1	0

A = 1

BC \ DE	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	1	1	0
10	0	1	1	0

Find the SoP from the 5-variable K-map.

A = 0

BC \ DE	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	1	1	0
10	0	1	1	0

A = 1

BC \ DE	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	1	1	0
10	0	1	1	0

$$F = BE + B'C'E'$$

Queen Mary
University of London

General Procedure: Designing Combinational Circuits

1. Specification: Write specification for the circuit if not already available.
   - Specify/label input(s) and output(s).
2. Formulation: Derive *Truth Table* or *initial Boolean equations* defining the relationships between inputs and outputs, if not in the specification.
3. Optimisation: Minimise the design using *Switching Algebra*, *Karnaugh Map*, *software*.
   - Draw logic diagram for the resulting circuit using AND/ OR/ NOT gates.
4. Technology Mapping: Map the logic diagram to the implementation technology selected (e.g., map into NANDs).
5. Verification: Verify the correctness of the final design *manually* or *using simulation*.

Practical Considerations:
- Cost of gates (Number)
- Maximum allowed delay
- Fan-in/Fan-out

Example: Circuit Design (1/2)

- *Question*: Design a circuit that has a 3-bit input and a single output (F) specified as follows:
  - F = 0, when the input is less than $(5)_{10}$
  - F = 1, otherwise

Step 1: Specification

- Label the inputs (3 bits) as X, Y, Z: *X* is the most significant bit, *Z* is the least significant bit.
- The output (1 bit) is F:
  - F = 1 → $(101)_2$, $(110)_2$, $(111)_2$
  - F = 0 → other inputs

Example: Circuit Design (2/2)

X	Y	Z	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

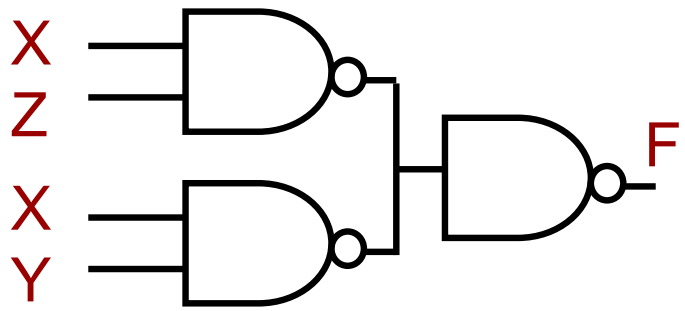
Step 2: Formulation

- Obtain the Truth Table. →

Step 3: Optimisation

X \ YZ	00	01	11	10
0	0	0	0	0
1	0	1	1	1

F = XZ + XY

Step 4: Technology mapping *(e.g., NAND implementation)*

- F = F'' = (XZ + XY)'' = ((XZ)'.(XY)')',
  applying *De Morgan's theorem*